Python & Anaconda Code Refresher

Baseball Data Edition: Getting Started From Scratch

A starter guide and cheat sheet for baseball people learning to work with data in Python. Part 1 explains the tools and gets you set up. Part 2 is the refresher: the code you will use most, so you do not have to memorize it. Every example uses free, public MLB pitch data from Baseball Savant.

Put together by Louis G. Messina with AI assistance, for his own study and for anyone starting out. Open source: suggestions welcome on GitHub. Every code example was tested on public Baseball Savant data (May 11-17, 2024).

How to read Part 2: each step has a short title, a blue label for where to type it, a gray box with exactly what to type, and a tip. "Anaconda Prompt" means the black command window (Terminal on a Mac). "Jupyter cell" means a cell in your notebook.

Part 1: Getting Started From Scratch

What the tools are

Tool	What it is	When you use it
Python	A free programming language. You write short instructions and the computer runs them.	Data that is too big, messy or repetitive for Excel.
Anaconda Distribution	A free installer that bundles Python, Jupyter and the common data packages in one download.	Setting up your computer once, without hunting for pieces.
Anaconda Navigator	Anaconda's point-and-click home screen.	Launching Jupyter without typing commands.
Anaconda Prompt (Windows) or Terminal (Mac)	A text window where you type commands.	Installing packages and turning environments on and off.
conda environment	A separate workspace with its own Python and packages.	Keeping one project's setup from breaking another's.
Jupyter Notebook / JupyterLab	A notebook that opens in your web browser. You type code in cells and the results appear right under each cell.	Exploring data step by step. This is where you will spend most of your time.
pandas	The Python package for tables (rows and columns), like Excel inside Python.	Loading, cleaning, filtering and summarizing data.
numpy and matplotlib	numpy does math on whole columns at once; matplotlib draws charts.	Calculations and charts.
pybaseball	A free package that pulls public Statcast data from Baseball Savant into pandas.	Getting pitch-by-pitch data without downloading files by hand.
Google Colab (optional)	Free Jupyter-style notebooks that run in your browser with a Google account. Nothing to install.	Trying Python before installing anything.
VS Code (optional, later)	A free code editor.	Longer .py scripts once notebooks feel comfortable.

Other tools you will hear about

Tool	Why it matters
Excel	Still the fastest way to look at a small table. Excel tops out at 1,048,576 rows per sheet, so full seasons of pitch data need Python.
SQL	The language for asking a database for data. Most teams store their data in databases, so SQL is often the first step before Python.
R and RStudio	Another data language, popular in baseball research. The ideas in this guide (filter, group, join, chart) carry over.
Baseball Savant	MLB's public Statcast site: every pitch, with speed, spin, movement and result. baseballsavant.mlb.com
FanGraphs and Baseball Reference	Season stats and leaderboards. Both let you export tables to CSV from the browser.

Install and open Python (about 20 minutes)

1. Download Anaconda. Go to anaconda.com/download, find Anaconda Distribution and pick your system (Windows or Mac). Choose Skip Registration if you do not want an account. It is free for personal use. Using it at work at an organization with more than 200 employees requires a paid license, so ask IT first.

2. Install it. Open the downloaded file and accept the defaults. Choose Just Me if asked. It needs several gigabytes of free space and takes a few minutes.

3. Open Anaconda Navigator. Windows: Start menu, type Anaconda Navigator. Mac: Applications folder. Navigator can take a minute to open the first time.

4. Launch Jupyter. In Navigator, click Launch under Jupyter Notebook (or JupyterLab). Your web browser opens to a list of your folders. Jupyter runs on your own computer; the browser is just the window.

5. Make a project folder. In the Jupyter file list, go to Documents, click New > New Folder, and rename it Baseball Data. Open it.

6. Start a notebook. Click New > Python 3 (ipykernel). A notebook opens. Click the title Untitled at the top and rename it, for example first_steps.

7. Run your first code. Click in the first cell, type `print("Hello, baseball")` and press Shift + Enter. The words appear under the cell. Then try `95.4 - 87.1` in the next cell. You just ran Python.

8. Save. Press Ctrl + S (Cmd + S on a Mac). Notebooks save as .ipynb files in your folder.

No install option: go to colab.research.google.com, sign in with a Google account and click New notebook. It works the same way. To use pybaseball there, run `!pip install pybaseball` in the first cell.

How a baseball data project flows

Every project, from a quick question to a front-office report, runs through the same six steps. Part 2 follows this order: set up, get data, look, clean, filter, build columns, summarize, check, chart and save.

Step	What it means
Ask	Write the question in one plain sentence. Example: Do relievers throw softer on the second of back-to-back days?
Get	Pull the data that can answer it, and note where it came from and when.
Check	Count rows, look for blanks, duplicates and impossible values before trusting anything.
Explore	Filter, group and chart. Baseball knowledge decides what to compare and what is fair.

Step	What it means
Answer	One clear table or chart, plus the caveats. Sample size matters.
Explain	Be able to walk someone through every step and say what you would do next.

Part 2: The Code Refresher

Gray boxes are what you type. Every Jupyter example below was run in order on one week of public Statcast data, so if you type them top to bottom in one notebook they will work.

1. ONE-TIME SETUP

1. Create a project environment (only once) [Anaconda Prompt]

```
conda create -n baseball python=3.11 -y
```

Tip: Creates a clean workspace named baseball. Mac users type these in Terminal.

2. Turn the environment on (every session) [Anaconda Prompt]

```
conda activate baseball
```

Tip: The prompt changes from (base) to (baseball). Do this before anything else.

3. Install the packages (only once) [Anaconda Prompt]

```
pip install pandas numpy matplotlib openpyxl pyarrow pybaseball scikit-learn jupyter
ipykernel
```

Tip: Takes a few minutes. openpyxl reads and writes Excel; pyarrow saves fast parquet files.

4. Let Jupyter find the environment (only once) [Anaconda Prompt]

```
python -m ipykernel install --user --name baseball
```

Tip: Now you can pick baseball as the kernel inside Jupyter.

5. See all environments [Anaconda Prompt]

```
conda env list
```

Tip: The active one has a * next to it.

2. EVERY SESSION

6. Go to your project folder [Anaconda Prompt]

```
cd "C:\Users\YourName\Documents\Baseball Data"
```

Tip: Shortcut: type cd and a space, then drag the folder into the window.

7. Open Jupyter [Anaconda Prompt]

```
jupyter notebook
```

Tip: Your browser opens. Click New > baseball. Keep the prompt window open while you work. jupyter lab also works.

8. Run a whole script instead of a notebook [Anaconda Prompt]

```
python my_script.py
```

Tip: Runs a .py file from top to bottom.

9. Close out when done [Anaconda Prompt]

```
Ctrl + C   (then)   conda deactivate
```

Tip: Save your notebook first. Ctrl + C stops Jupyter.

3. JUPYTER KEYBOARD SHORTCUTS

10. Run the current cell [Jupyter]

```
Shift + Enter
```

Tip: Output appears under the cell. [*] next to a cell means it is still running.

11. Insert a cell above / below [Jupyter]

```
Esc, then A   /   Esc, then B
```

Tip: Esc puts you in command mode (blue bar on the left).

12. Turn a cell into notes text [Jupyter]

```
Esc, then M
```

Tip: Use notes cells for headings and to explain each step in plain English.

13. Delete a cell [Jupyter]

```
Esc, then D, D
```

Tip: Press D twice. Esc, then Z undoes it.

14. Finish a name or see help while typing [Jupyter]

```
Tab   /   Shift + Tab
```

Tip: Tab completes column and function names. Shift + Tab shows what a function expects.

15. Where am I, and what files are here? [Jupyter cell]

```
%pwd
%ls
```

Tip: Confirms your data file is in the current folder.

16. Fresh start if things get weird [Jupyter menu]

```
Kernel > Restart & Run All
```

Tip: Reruns everything from the top. The best check that your notebook really works.

4. PYTHON BASICS IN FIVE MINUTES

17. Store a value in a variable [Jupyter cell]

```
velo = 95.4
pitcher = "Paul Skenes"
print(pitcher, velo)
```

Tip: Text goes in quotes; numbers do not. Names are case-sensitive: velo and Velo are different.

18. A list holds several values in order [Jupyter cell]

```
fastballs = ["FF", "SI", "FC"]
print(fastballs[0], len(fastballs))
```

Tip: Counting starts at 0, so fastballs[0] is the first item.

19. A dictionary looks things up [Jupyter cell]

```
pitch_names = {"FF": "4-Seam Fastball", "SL": "Slider", "CH": "Changeup"}
print(pitch_names["SL"])
```

Tip: Key on the left, value on the right. Like a two-column lookup table.

20. Make a decision with if [Jupyter cell]

```
if velo >= 95:
    print("Plus velocity")
else:
    print("Below 95")
```

Tip: The colon and the indent (4 spaces) matter. Python uses indentation instead of brackets.

21. Repeat with a for loop [Jupyter cell]

```
for p in fastballs:
    print(p, "is a fastball code")
```

Tip: Runs the indented line once for each item in the list.

22. Mix text and numbers with an f-string [Jupyter cell]

```
gap = 95.4 - 87.1
print(f"Velo gap: {gap:.1f} mph")
```

Tip: :.1f rounds to one decimal place.

5. GET PUBLIC BASEBALL DATA

23. Download from Baseball Savant by hand [Web browser]

```
baseballsavant.mlb.com/statcast_search
```

Tip: Pick season, dates or a player, click Search, then the CSV download icon. Very large searches can be cut short, so pull a week or a month at a time.

24. Pull a date range with pybaseball [Jupyter cell]

```
from pybaseball import statcast
df = statcast(start_dt="2024-05-11", end_dt="2024-05-17")
df.shape
```

Tip: One week of 2024 regular-season games is about 27,000 pitches. Bigger ranges take longer, so start small.

25. Find a player's ID [Jupyter cell]

```
from pybaseball import playerid_lookup
playerid_lookup("skenes", "paul")
```

Tip: Use the key_mlbam number. IDs never change; names can be shared or spelled differently.

26. Pull one pitcher's pitches [Jupyter cell]

```
from pybaseball import statcast_pitcher
sk = statcast_pitcher("2024-04-01", "2024-09-30", player_id=694973)
sk.shape
```

Tip: 694973 is Paul Skenes. Swap in any key_mlbam.

27. Save it so you do not pull it twice [Jupyter cell]

```
df.to_csv("statcast_2024_05_11_17.csv", index=False)
```

Tip: index=False keeps an extra row-number column out of the file.

6. LOAD THE DATA

28. Import the core packages (first cell, always) [Jupyter cell]

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

Tip: pd, np and plt are the standard nicknames everyone uses.

29. Read a CSV file [Jupyter cell]

```
df = pd.read_csv("statcast_2024_05_11_17.csv")
```

Tip: The file must be in the current folder, or give the full path.

30. Read an Excel sheet [Jupyter cell]

```
# stats = pd.read_excel("my_file.xlsx", sheet_name="Sheet1")
```

Tip: Slow for big files. Shown as a comment (#) so it does not run until you have the file.

31. Save and reload a fast copy [Jupyter cell]

```
df.to_parquet("pitches.parquet")
df = pd.read_parquet("pitches.parquet")
```

Tip: Parquet loads in seconds and keeps column types. Use it for big files.

32. Show all the columns [Jupyter cell]

```
pd.set_option("display.max_columns", 120)
```

Tip: Savant files have over 100 columns; by default pandas hides the middle ones.

7. LOOK AT THE DATA

33. Rows and columns [Jupyter cell]

```
df.shape
```

Tip: Returns (rows, columns). The first thing to check every time.

34. First and last rows [Jupyter cell]

```
df.head()
df.tail(3)
```

Tip: df.head(20) shows 20 rows.

35. List the column names [Jupyter cell]

```
df.columns.tolist()
```

Tip: Copy names exactly from here to avoid KeyError typos. Savant column meanings: baseballsavant.mlb.com/csv-docs

36. Types and blanks [Jupyter cell]

```
df.info()
df.isna().sum().sort_values(ascending=False).head(10)
```

Tip: launch_speed and launch_angle are blank unless the ball was put in play. That is expected.

37. Summary stats [Jupyter cell]

```
df[["release_speed", "release_spin_rate", "pfx_x", "pfx_z"]].describe()
```

Tip: Check min and max for impossible values. pfx_x and pfx_z are movement in feet.

38. Count each value [Jupyter cell]

```
df["pitch_type"].value_counts()
```

Tip: Savant codes: FF 4-seam, SI sinker, FC cutter, SL slider, ST sweeper, CU curve, KC knuckle curve, CH changeup, FS splitter.

39. How many different pitchers? [Jupyter cell]

```
df["pitcher"].nunique()
```

Tip: In Savant files, pitcher is the ID and player_name is the pitcher's name (Last, First).

8. CLEAN THE DATA

40. Make dates real dates [Jupyter cell]

```
df["game_date"] = pd.to_datetime(df["game_date"])
```

Tip: Now you can filter by date and pull out months or weekdays.

41. Convert Excel serial dates [Jupyter cell]

```
pd.to_datetime(45550, unit="D", origin="1899-12-30")
```

Tip: Excel stores dates as day counts. 45550 becomes 2024-09-15.

42. Keep only the columns you need [Jupyter cell]

```
cols = ["game_pk", "game_date", "pitcher", "player_name", "p_throws", "stand",
        "inning", "inning_topbot", "home_team", "away_team", "at_bat_number",
        "pitch_number", "balls", "strikes", "pitch_type", "pitch_name",
        "release_speed", "release_spin_rate", "pfx_x", "pfx_z", "description",
        "events", "launch_speed", "launch_angle", "game_type"]
pitches = df[cols].copy()
```

Tip: .copy() makes a separate table, which avoids the SettingWithCopyWarning later.

43. Drop rows missing key values [Jupyter cell]

```
pitches = pitches.dropna(subset=["pitch_type", "release_speed"])
```

Tip: A few pitches each week have no pitch type or speed (tracking hiccups).

44. Remove exact duplicates [Jupyter cell]

```
before = len(pitches)
pitches = pitches.drop_duplicates()
print(before - len(pitches), "duplicates removed")
```

Tip: Always print how many rows a cleaning step removed.

45. Rename a column [Jupyter cell]

```
pitches = pitches.rename(columns={"player_name": "pitcher_name"})
```

Tip: Clear names make your notebook easier to read later.

46. Turn text into numbers [Jupyter cell]

```
pitches["release_speed"] = pd.to_numeric(pitches["release_speed"], errors="coerce")
```

Tip: errors="coerce" turns anything that is not a number into a blank instead of crashing.

9. FILTER AND SELECT

47. Regular season only [Jupyter cell]

```
pitches = pitches[pitches["game_type"] == "R"]
```

Tip: Use == to test equals. One = assigns a value.

48. One pitcher, by ID [Jupyter cell]

```
one = pitches[pitches["pitcher"] == 694973]
```

Tip: Always filter by ID, not name.

49. Two conditions (and = &, or = |) [Jupyter cell]

```
lefty_sl = pitches[(pitches["p_throws"] == "L") & (pitches["pitch_type"] == "SL")]
```

Tip: Wrap each condition in parentheses. Forgetting them is the most common error.

50. Match any value in a list [Jupyter cell]

```
FASTBALLS = ["FF", "SI", "FC"]
fb = pitches[pitches["pitch_type"].isin(FASTBALLS)]
```

Tip: isin is cleaner than a chain of ors.

51. A date range [Jupyter cell]

```
early = pitches[pitches["game_date"].between("2024-05-11", "2024-05-13")]
```

Tip: Works once game_date is a real date (section 8).

52. Which team is pitching? [Jupyter cell]

```
pitches["pitching_team"] = np.where(pitches["inning_topbot"] == "Top",
                                    pitches["home_team"], pitches["away_team"])
```

Tip: Baseball logic in code: the home team pitches in the top of the inning. np.where works like Excel's IF.

53. Pick a few columns [Jupyter cell]

```
pitches[["pitcher_name", "pitching_team", "pitch_type", "release_speed"]].head()
```

Tip: Double brackets mean a list of columns.

10. CREATE NEW COLUMNS

54. Flag swings and whiffs [Jupyter cell]

```
SWINGS = ["swinging_strike", "swinging_strike_blocked", "foul", "foul_tip",
          "hit_into_play", "foul_bunt", "missed_bunt", "bunt_foul_tip"]
WHIFFS = ["swinging_strike", "swinging_strike_blocked"]
pitches["swing"] = pitches["description"].isin(SWINGS)
pitches["whiff"] = pitches["description"].isin(WHIFFS)
```

Tip: Sites define whiffs slightly differently (foul tips, bunts). Write your definition down and stick to it.

55. Build the count [Jupyter cell]

```
pitches["count"] = pitches["balls"].astype(str) + "-" + pitches["strikes"].astype(str)
```

Tip: astype(str) turns numbers into text so they can be joined: 3-2, 0-1 and so on.

56. Movement in inches, arm side positive [Jupyter cell]

```
pitches["hb_in"] = pitches["pfx_x"] * 12
pitches["ivb_in"] = pitches["pfx_z"] * 12
# arm-side break is positive for every pitcher
pitches["hb_arm_side"] = np.where(pitches["p_throws"] == "R",
                                  -pitches["hb_in"],
                                  pitches["hb_in"])
```

Tip: pfx_x is from the catcher's view, so righties' arm-side pitches start negative. Flipping them puts every pitcher on one scale: arm side positive, glove side negative.

57. Group pitch codes with a lookup [Jupyter cell]

```
GROUPS = {"FF": "Fastball", "SI": "Fastball", "FC": "Fastball",
          "SL": "Breaking", "ST": "Breaking", "CU": "Breaking",
          "KC": "Breaking", "SV": "Breaking",
          "CH": "Offspeed", "FS": "Offspeed"}
pitches["pitch_group"] = pitches["pitch_type"].map(GROUPS).fillna("Other")
```

Tip: map uses the dictionary from section 4. Codes not in it (knuckleball, eephus) become Other.

58. Number each pitcher's pitches in a game [Jupyter cell]

```
pitches = pitches.sort_values(["game_pk", "at_bat_number", "pitch_number"])
pitches["pitch_of_game"] = pitches.groupby(["game_pk", "pitcher"]).cumcount() + 1
```

Tip: Sort first so pitches are in the order they were thrown. Useful for fatigue questions.

11. GROUP AND SUMMARIZE

59. Average velo by pitcher and pitch type [Jupyter cell]

```
velo = (pitches.groupby(["pitcher", "pitcher_name", "pitch_type"])["release_speed"]
          .agg(n="count", avg_velo="mean").reset_index())
```

Tip: reset_index() turns the groups back into normal columns.

60. Set a minimum sample [Jupyter cell]

```
velo = velo[velo["n"] >= 30]
```

Tip: Small samples mislead. Pick a minimum before you look at the results.

61. Sort and show the top 10 [Jupyter cell]

```
velo.sort_values("avg_velo", ascending=False).head(10)
```

Tip: ascending=True (the default) sorts smallest first.

62. Whiff rate by pitch type [Jupyter cell]

```
sw = pitches[pitches["swing"]]
whiff_rate = sw.groupby("pitch_type")["whiff"].agg(swings="count", whiff_pct="mean")
whiff_rate.sort_values("whiff_pct", ascending=False)
```

Tip: The mean of True/False is the share that are True, so mean() gives the whiff rate.

63. Pitch mix with a pivot table [Jupyter cell]

```
mix = pd.crosstab(pitches["pitcher_name"], pitches["pitch_type"],
normalize="index").round(3)
mix.loc[["Skenes, Paul"]]
```

Tip: Each row adds up to 1, so you read it as percent of that pitcher's pitches. Works like an Excel pivot table.

64. Attach a summary back to each row (merge) [Jupyter cell]

```
totals = (pitches.groupby(["game_pk", "pitcher"]).size()
          .reset_index(name="pitches_in_game"))
pitches = pitches.merge(totals, on=["game_pk", "pitcher"], how="left")
```

Tip: merge is Python's VLOOKUP. how="left" keeps every pitch even if no match is found.

12. DATA CHECKS

65. Impossible counts [Jupyter cell]

```
pitches[(pitches["balls"] > 3) | (pitches["strikes"] > 2)]
```

Tip: Should be empty or nearly empty.

66. Suspiciously slow pitches [Jupyter cell]

```
pitches[pitches["release_speed"] < 60][["pitcher_name", "pitch_type", "release_speed"]]
```

Tip: Usually position players pitching or eephus pitches. Decide whether to keep them.

67. Handedness values [Jupyter cell]

```
pitches["p_throws"].value_counts()
```

Tip: Anything besides L and R needs a look.

68. One name, more than one ID? [Jupyter cell]

```
ids = pitches.groupby("pitcher_name")["pitcher"].nunique()
ids[ids > 1]
```

Tip: Over several seasons you will see shared names (there have been multiple pitchers named Luis Garcia). Another reason to use IDs.

69. Pitches per day [Jupyter cell]

```
pitches.groupby("game_date").size()
```

Tip: A day with far fewer pitches than the others can mean missing games in your pull.

13. CHARTS

70. Histogram of 4-seam velocity [Jupyter cell]

```
pitches.loc[pitches["pitch_type"] == "FF", "release_speed"].hist(bins=40)
plt.xlabel("Release speed (mph)")
plt.ylabel("Pitches")
plt.title("4-Seam Fastball Velocity")
plt.show()
```

Tip: Always label axes. A chart without units is a guess.

71. Bar chart of whiff rate [Jupyter cell]

```
whiff_rate["whiff_pct"].sort_values().plot(kind="barh")
plt.xlabel("Whiff rate on swings")
plt.show()
```

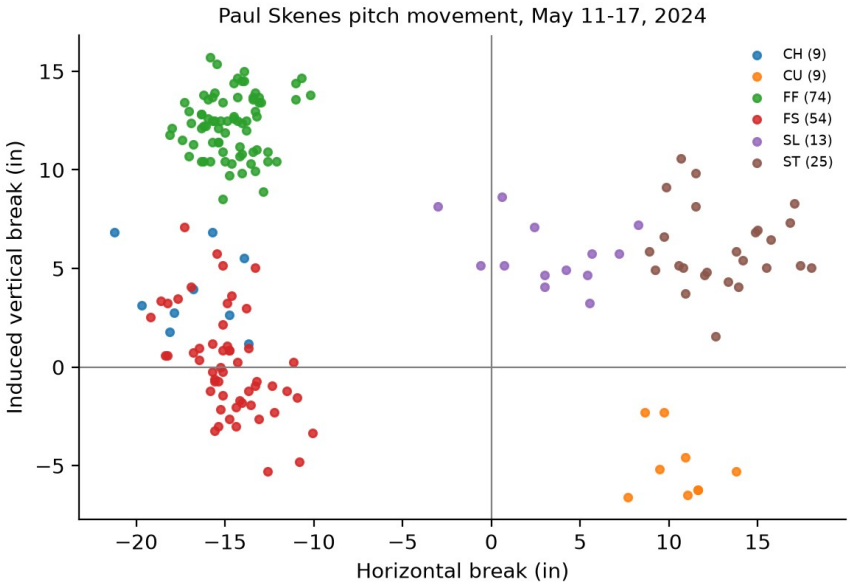
Tip: barh makes horizontal bars, easier to read with labels.

72. Movement chart for one pitcher [Jupyter cell]

```
one = pitches[pitches["pitcher"] == 694973]
for pt, g in one.groupby("pitch_type"):
    plt.scatter(g["hb_in"], g["ivb_in"], s=8, label=pt)
plt.axhline(0, color="gray"); plt.axvline(0, color="gray")
plt.xlabel("Horizontal break (in)"); plt.ylabel("Induced vertical break (in)")
plt.legend(); plt.show()
```

Tip: One color per pitch type. Uses the inch columns from section 10.

What you should see (Paul Skenes, May 11-17, 2024):



Savant measures horizontal break from the catcher's view, so a righty's arm-side pitches (4-seamer, splitter) show as negative.

73. Velo by pitch of game [Jupyter cell]

```
ff = pitches[pitches["pitch_type"] == "FF"]
ff.groupby("pitch_of_game")["release_speed"].mean().loc[1:100].plot()
plt.xlabel("Pitch of game"); plt.ylabel("Avg 4-seam velo (mph)")
plt.show()
```

Tip: Careful: later pitch numbers include only pitchers who stayed in, mostly starters. That is a selection effect, not just fatigue.

74. Save a chart as an image [Jupyter cell]

```
plt.savefig("my_chart.png", dpi=200, bbox_inches="tight")
```

Tip: Put this line before plt.show(). dpi=200 makes it sharp enough for slides.

14. SAVE YOUR WORK

75. Save a table to CSV [Jupyter cell]

```
velo.to_csv("pitcher_velo.csv", index=False)
```

Tip: CSV opens in Excel, Google Sheets and anything else.

76. Several tables in one Excel file [Jupyter cell]

```
with pd.ExcelWriter("weekly_summary.xlsx") as xw:
    velo.to_excel(xw, sheet_name="Velo", index=False)
    whiff_rate.to_excel(xw, sheet_name="Whiffs")
```

Tip: One sheet per table. Needs openpyxl (installed in section 1).

77. Save the cleaned pitch data [Jupyter cell]

```
pitches.to_parquet("pitches_clean.parquet")
```

Tip: Keep the raw download untouched and save cleaned versions under new names.

15. COMMON ERRORS AND FIXES

78. ModuleNotFoundError: No module named ... [Anaconda Prompt]

```
conda activate baseball
pip install <package name>
```

Tip: Usually the wrong environment or kernel. In Jupyter: Kernel > Change kernel > baseball.

79. FileNotFoundError [Jupyter cell]

```
import os
os.getcwd()
os.listdir()
```

Tip: The file is not in this folder, or the name is spelled differently.

80. KeyError: 'column name' [Jupyter cell]

```
df.columns.tolist()
```

Tip: A typo or wrong capitalization. Python is case-sensitive.

81. NameError: name '...' is not defined [Jupyter menu]

```
Kernel > Restart & Run All
```

Tip: You ran cells out of order, or skipped the cell that created it.

82. SettingWithCopyWarning [Jupyter cell]

```
subset = df[df["pitch_type"] == "SL"].copy()
```

Tip: Add .copy() when you make a filtered table you plan to change.

83. TypeError when doing math [Jupyter cell]

```
df["release_speed"] = pd.to_numeric(df["release_speed"], errors="coerce")
```

Tip: The column is stored as text. Convert it to numbers first.

84. Ask Python for help on anything [Jupyter cell]

```
help(pd.DataFrame.groupby)
```

Tip: Or type a ? after any name, like df.groupby? and run the cell.

Good habits

- Write the question in plain English at the top of the notebook.
- Never edit the raw download. Save cleaned data under a new name.
- Print row counts before and after every filter or cleaning step.
- Explain each step in a notes cell (Esc, then M).
- Run Kernel > Restart & Run All before you share anything.
- Check one or two of your numbers against Baseball Savant's own leaderboards.
- If AI helped you, say what it did. Only share work you can explain line by line.

Data: public Statcast data from Baseball Savant (baseballsavant.mlb.com), pulled with pybaseball. Column definitions: baseballsavant.mlb.com/csv-docs. Anaconda terms: anaconda.com/legal/terms/terms-of-service.